ESD PLATFORM

이벤트 기반 게임 스토어 백엔드 — 8–9월 구현의 문제 해결

신지섭 · 개인 프로젝트 · 2026.08 · Java 21 / Spring Boot 3.5 / Apache Kafka

개요

8–9월 ESD 구현 — 10월 MVP 재작성 이전

게임 등록과 심사, 구매 · 결제 · 정산 기능을 **서비스 아홉과 게이트웨이**로 나눠 하나의 저장소에서 개발했습니다.
서비스 간 통신에는 **Kafka 이벤트**를 사용하고, 이벤트 발행과 중복 처리 방지를 위해 **Outbox / Inbox 패턴**을 적용했습니다.

겪은 문제와 결과

1. 정상 결제 자동 환불 — 환불 8 → 0건

장애 주입 테스트에서 `license`의 DB 접근을 60초 차단하자 정상 결제 8건이 환불됐습니다. 재시도 소진만으로 환불하던 로직에 저장소 장애와 지급 여부 확인을 추가했습니다.

2. Outbox 발행 순서 역전 — 역전 0건 · 처리량 ×1.13

발행에 실패한 이벤트를 건너뛰면서 결제 취소가 결제 완료보다 먼저 전달됐습니다. 같은 주문의 이벤트는 순서대로 발행하고, 다른 주문은 병렬로 처리하도록 변경했습니다.

3. Kafka 구간 추적 단절 — 추적 일원화 · p95 절반

Outbox 발행 시점에 요청의 추적 정보가 없어 서비스 간 추적이 끊겼습니다. 추적 정보를 이벤트와 함께 저장하고 발행 시 헤더로 전달했습니다. 연결된 트레이스에서 확인한 폴링 대기도 줄였습니다.

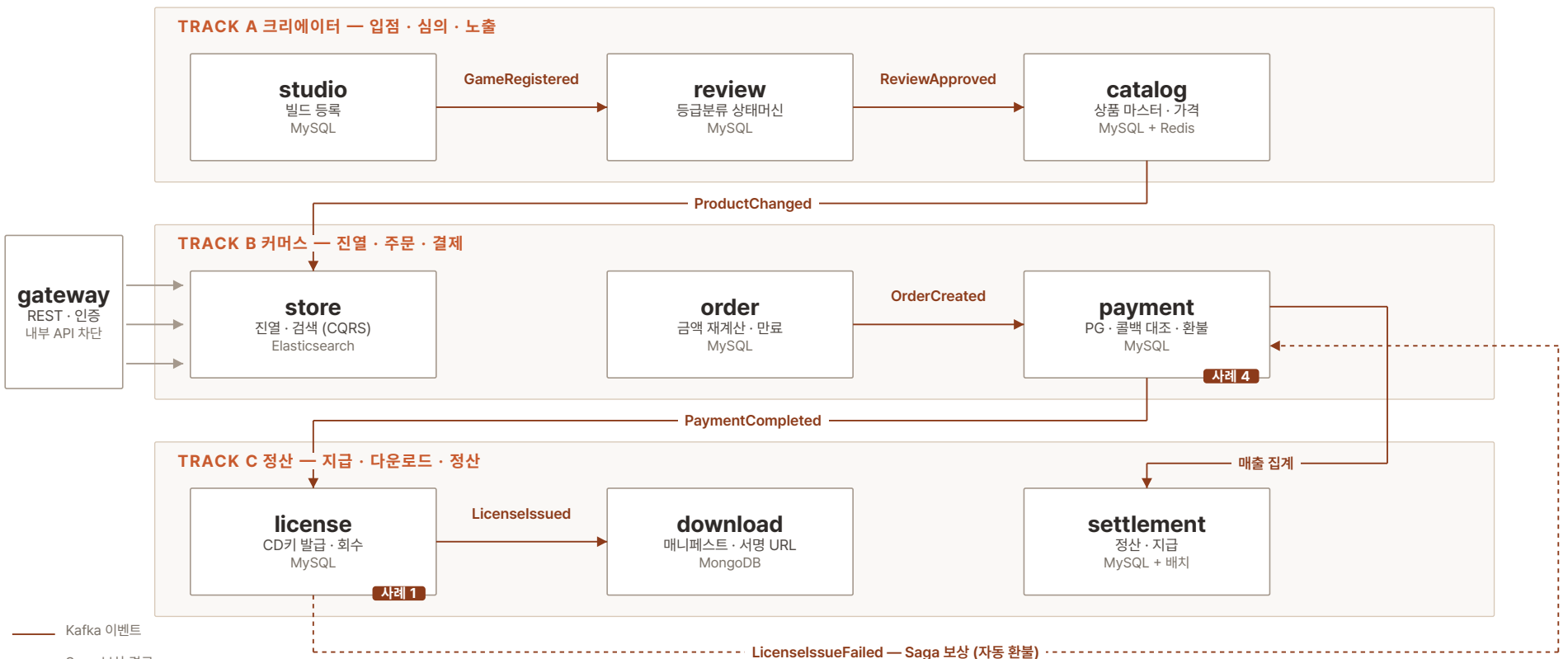
4. 중복으로 오인된 결제 승인 — 승인 유실 0건

결제사의 멱등키로 결제를 조회해, 키가 재사용되면 다른 주문의 승인을 중복으로 오인했습니다. 주문번호로 결제를 조회하도록 바꾸고 중복 판정과 동시성 제어를 분리했습니다.

언어 · 프레임워크	Java 21 · Spring Boot 3.5 · Spring Cloud Gateway · Apache Kafka (KRaft) · JPA/Hibernate · Flyway · MySQL 8 · Gradle 멀티모듈
분산 정합성	Transactional Outbox · Inbox 멱등 가드 · Saga 보상 트랜잭션 · DLT 재처리 · CQRS
테스트	ArchUnit (앱당 36규칙) · Testcontainers · JUnit 5 · PIT 뮤테이션 테스트 · OpenAPI 스냅샷 계약 테스트
관측 · 알람	Micrometer Tracing + OTLP · Grafana Tempo · Prometheus (알람 11종) · kafka-exporter · k6 부하 시나리오 4종 · 장애 주입 하네스
인프라 · CI	Docker Compose (인프라 11 + 앱 10) · Redis · Elasticsearch · MongoDB · GitHub Actions self-hosted 러너 · devcontainer

아키텍처

서비스 간 이벤트 흐름



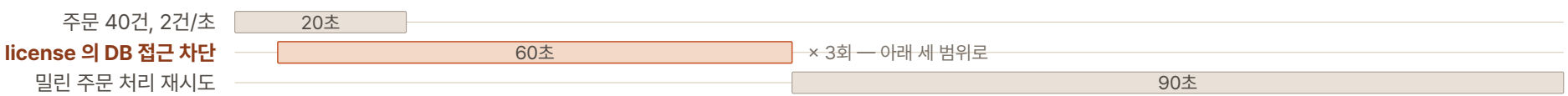
문제 해결

문제 → 해결 → 결과 로 정리했습니다.
각 사례는 테스트로 문제를 재현한 뒤 수정 결과를 확인했습니다.

1 정상 결제 자동 환불 · 저장소 장애를 지급 실패로 판단한 자동 환불
2 Outbox 발행 순서 역전 · 발행 실패 후 후행 이벤트가 먼저 전달되는 문제
3 Kafka 구간 추적 단절 · 요청 스레드 밖에서 발행할 때 추적 정보가 유실되는 문제
4 중복으로 오인된 결제 승인 · 멱등키를 결제 조회에도 사용한 콜백 처리

Saga 자동 환불 오작동 — 장애 주입 테스트에서 정상 결제 8건 환불

결제와 라이선스 지급은 서로 다른 서비스에서 처리합니다. 하나의 트랜잭션으로 묶을 수 없어 지급이 실패하면 자동 환불하도록 구현했습니다. 컨슈머는 7초 동안 최대 네 번 시도하므로, DB 장애가 길어지면 정상 결제도 재시도 소진으로 환불될 수 있다고 예상했습니다. 이를 확인하기 위해 부하를 건 상태에서 license 서비스의 DB 접근을 차단했습니다.



지급 결과는 `license`, 보상 이벤트는 `outbox_event` 테이블에 저장합니다. 접근 권한을 회수해 아래 세 조건을 각각 한 회차씩 테스트했습니다.

차단 범위	지급 완료	보류	환불
없음	40	0	0
데이터베이스 전체 — `license` 와 `outbox_event` 둘 다	39	1	0
`license` **테이블만** — `outbox_event` 는 정상	32	0	**8**

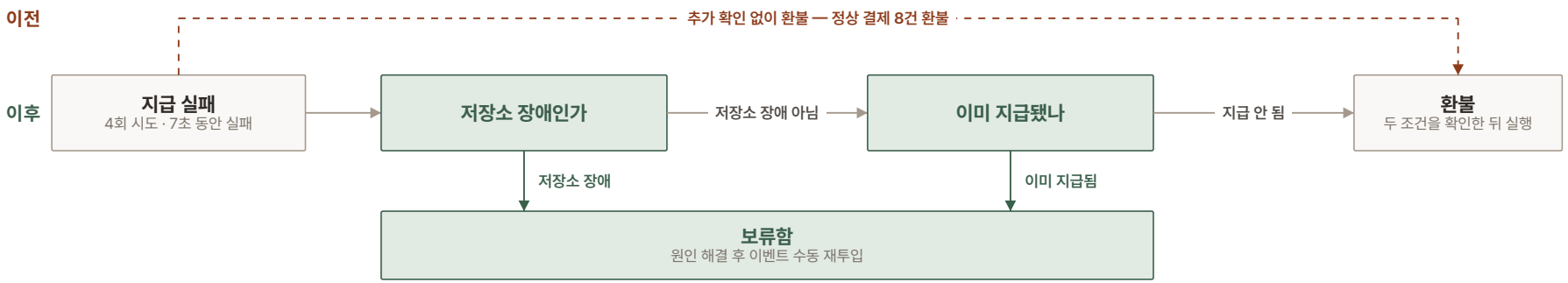
보류는 결제 후 라이선스가 아직 지급되지 않은 상태입니다. 데이터베이스 전체를 차단하면 보상 이벤트도 저장할 수 없어 환불이 실행되지 않습니다. `outbox_event`에 접근할 수 있는 조건에서만 보상 이벤트가 발행돼 8건이 환불됐습니다. 40건 중 8건만 환불된 이유는 블로킹 재시도 중 파티션 처리가 7초씩 지연되기 때문입니다.

자동 환불 조건을 **재시도 소진만으로 판단한 것**이 문제였습니다. 저장소 장애로 지급 결과를 확인할 수 없는 상황까지 지급 실패로 처리했습니다. 여덟 건 모두 **60초 뒤 DB 접근이 복구되면 지급할 수 있는 정상 결제**였습니다. 환불 처리 자체는 정산상 정상으로 기록되므로 대사에서도 발견되지 않습니다.

데이터베이스 전체를 차단한 회차에서는 같은 주문번호에 지급 성공과 최종 실패 로그가 함께 남았습니다. 오프셋 커밋 전 연결이 끊겨 메시지가 재배달됐고, 재처리 실패로 보상이 시작됐습니다. 이 회차에서는 보상 이벤트를 저장할 수 없어 환불이 실행되지 않았습니다. 하지만 `license` 테이블만 차단됐다면 **이미 지급된 라이선스를 유지한 채 결제가 환불되는 상태**가 됩니다.

문제 해결 결과

환불 조건에 저장소 장애와 지급 여부 확인 추가



컨슈머는 최초 처리 후 1 → 2 → 4초 간격으로 세 번 재시도합니다. 재시도를 소진하면 **저장소 장애인지, 이미 지급된 주문인지** 확인한 뒤 환불하도록 변경했습니다. 지급 여부 조회도 DB 접근이 필요하므로 저장소 장애를 먼저 판별합니다.

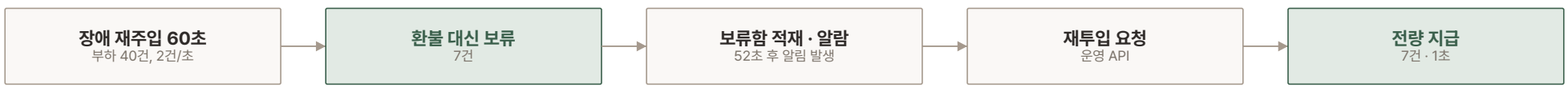
장애 주입 중 발생한 트랜잭션 예외도 판별할 수 있도록 데이터 접근 · 트랜잭션 · JDBC 예외를 모두 검사했습니다. 지급 여부는 환불 직전에 `license` 테이블을 조회해 확인합니다. **Kafka에서는 같은 메시지가 재배달될 수 있으므로**, 저장소가 정상이어도 재처리 실패만으로 환불해서는 안 됩니다.

저장소 장애이거나 이미 지급된 주문은 자동 환불하지 않고 보류하도록 변경했습니다. 보류 건이 쌓이면 알림을 보내고, 원인을 해결한 뒤 수동으로 재처리합니다.

검토한 대안	채택하지 않은 이유
재시도를 더 오래 한다	블로킹 재시도 중에는 파티션 처리가 멈춥니다. 재시도를 60초 유지하면 후속 정상 주문도 60초 지연됩니다
성공할 때까지 무한 재시도한다	이벤트 규약에 맞지 않는 메시지는 반복해도 실패합니다. 오프셋이 진행되지 않아 후속 정상 메시지도 처리할 수 없습니다
보류함에 리스너를 두고 자동 재투입한다	원인을 해결하지 않고 재투입하면 같은 오류로 다시 보류됩니다. 반복 실패를 막기 위해 재투입은 수동으로 실행하도록 했습니다

불필요한 환불 8 → 0건 — 보류 7건은 1초 내 전량 지급

동일한 조건으로 장애를 다시 주입하고, 보류된 주문이 재처리 후 정상 지급되는지 확인했습니다.



주문 40건의 처리 결과	수정 전	수정 후
정상 지급	32	33
보류 — 재처리 가능	0	**7**
불필요한 환불 — 되돌릴 수 없음	**8**	**0**

보류로 남은 **7건은 재투입 요청 한 번으로 1초 안에** 전부 지급됐습니다. 이후 조건마다 2회씩 재측정하고, 각 회차 직전에 장애 없는 조건을 실행해 결과를 비교했습니다. DB 접근을 15분간 막은 회차는 두 번 다 **800건에 환불 0건**이었습니다(보류 47건 · 52건). 장애 없는 회차는 800건 전부 정상이었습니다.

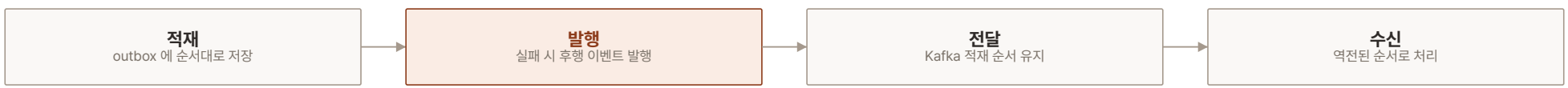
이미 지급된 주문의 환불 방지
데이터베이스 전체를 차단한 테스트에서는 지급 트랜잭션 커밋 후 Kafka 오프셋 커밋 전에 연결이 끊겼습니다. 이미 지급된 주문의 메시지가 재배달됐고, 재처리 실패로 보상이 시작됐습니다. 이를 막기 위해 환불 직전에 지급 여부를 조회하도록 변경했습니다. 이미 지급됐거나 저장소 장애로 지급 여부를 확인할 수 없는 주문은 보류하고, 그 외 지급 실패에 대해서만 자동 환불합니다.

환불 8 → 0
정상 결제의 불필요한 환불 방지

4,153 : 0
누적 license 이벤트 대비 지급 실패

Outbox 발행 순서 역전 — 환불된 주문에 남는 라이선스

같은 주문의 결제 완료 이벤트는 라이선스를 발급하고, 결제 취소 이벤트는 라이선스를 회수합니다. 처리 순서가 바뀌면 최종 지급 상태도 달라집니다.
발행 재시도에 백오프를 적용하던 중 두 이벤트의 발행 순서가 뒤바뀔 수 있음을 발견했습니다.



Outbox 적재 순서와 Kafka 전달 순서는 정상이었습니다. 순서가 바뀌는 지점은 **Outbox 릴레이에서 Kafka로 발행하는 과정**이었습니다. 주문번호를 메시지 키로 사용해도 Kafka에 적재되기 전에 바뀐 순서까지 복구할 수는 없습니다.

> **컨슈머는 결제 완료 후 취소가 도착한다고 가정했습니다**
> 라이선스 서비스는 회수할 라이선스가 없으면 이미 회수된 것으로 판단해 처리를 종료합니다. 정산도 같은 방식으로 처리합니다. 따라서 회수 이벤트가 먼저 도착하면 예외나 경고 없이 종료되고, 뒤늦게 도착한 결제 완료 이벤트가 **환불된 주문에 라이선스를 발급**합니다.

같은 주문에 도착한 순서	결제	라이선스
결제 완료 → 결제 취소	환불됨	회수됨
결제 취소 → 결제 완료 (역전)	환불됨	**유효**

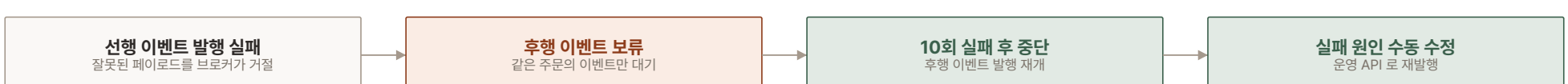
순서가 역전되면 **결제는 환불됐지만 라이선스는 유효한 상태**가 됩니다. 평소에는 결제 완료가 발행된 뒤 취소 이벤트가 적재돼 두 이벤트가 겹치지 않습니다. Outbox에 두 이벤트가 모두 미발행 상태로 남아 있으면 같은 배치에서 조회될 수 있습니다. 이때 완료 이벤트의 발행에 실패해도 취소 이벤트를 보내면 순서가 역전됩니다.

발행 순서를 보장할 범위와 대안 비교

같은 주문의 이벤트 순서를 보장
배치 전체를 직렬화하면 다른 주문까지 지연됩니다. 순서가 지급 결과에 영향을 주는 범위는 **동일한 주문의 이벤트**이므로, 주문번호를 기준으로 직렬화하기로 했습니다.

어디서	방법	하는 일	판단
보내는 쪽	전면 중단	실패하면 배치 전체를 멈춘다	한 건의 실패로 무관한 주문도 지연
	키 단위 직렬화	같은 주문의 후행 이벤트만 보류	**채택.** 주문 내 순서 보장 · 주문 간 병렬 처리
	워커별 키 배정	한 키는 늘 같은 릴레이가 맡는다	다중 릴레이에 필요. 현재는 1대로 운영
DB 로그	CDC (Debezium)	폴링 대신 DB 로그를 읽어 발행	발행을 커넥터에 위임. 이번에는 릴레이를 직접 개선
받는 쪽	컨슈머가 재정렬	번호를 싣고, 어긋나면 보류	컨슈머별 버퍼 · 타임아웃 관리 필요. 선행 이벤트 누락 시 대기
	낮은 버전 버리기	더 최신을 처리했으면 무시	이전 이벤트를 생략할 수 없는 지급 기록에 부적합
순서 의존 제거	순서 의존 제거	상태 변경 대신 현재 상태를 전달	지급 이벤트에 적용. 발급 · 회수는 순서 보장 필요

선행 이벤트가 계속 실패할 때의 처리



선행 이벤트가 실패하면 같은 주문의 후행 이벤트를 보류합니다. 주문 처리가 무기한 멈추지 않도록 시도 횟수에 상한을 뒀습니다. 상한에 도달하면 발행을 중단하고, 원인을 수정한 뒤 운영 API로 재발행합니다.

주문별 순차 발행과 주문 간 병렬 발행 적용

기존 릴레이는 이벤트를 한 건 발행할 때마다 브로커 응답을 기다렸습니다. 발행이 실패해도 다음 이벤트로 넘어가므로 같은 주문의 순서가 역전됐습니다. 다른 주문도 앞선 응답을 기다려야 해 **순서 보장과 처리량 모두 문제가 있었습니다.**

```
이전   A-1 보내고 기다림 → A-2 보내고 기다림 → B-1 보내고 기다림 → …
       A-1 이 실패해도 A-2 는 그대로 나간다        ← 같은 주문인데 순서가 뒤집힌다
       기다린 횟수 = 배치에 든 건수 (최대 200)

이후   [A-1, B-1, C-1]  한꺼번에 걸고 → 한 번 기다림
       [A-2,      C-2]  앞이 성공한 주문만 (B 는 이번 회차 보류)
       기다린 횟수 = 한 주문에 걸린 이벤트 수 (대개 1)
```

배치를 주문번호로 묶어 **서로 다른 주문은 병렬로 발행**하도록 변경했습니다. 같은 주문에서는 선행 이벤트의 발행이 성공한 뒤에만 다음 이벤트를 발행합니다. 실패 시에도 해당 주문의 후행 이벤트만 보류하므로 다른 주문을 계속 처리할 수 있습니다.

처리량이 함께 오른 이유
응답을 기다리는 범위를 같은 주문으로 제한하면서 **서로 다른 주문의 발행을 병렬로 처리**할 수 있게 됐습니다. 주문 내 순서를 보장하면서 배치 전체의 대기 시간도 줄었습니다.

1차 수정의 한계 — 배치가 나뉘면 발행 순서가 다시 역전됨

1차 수정 후, 선행 이벤트가 재시도를 기다리는 동안 같은 주문의 후행 이벤트가 발행되지 않는지 테스트를 추가했습니다. 테스트 결과, **두 이벤트가 다른 배치로 조회되면 순서가 다시 역전됐습니다.**

릴레이는 다음 시도 시각이 없거나 이미 지난 이벤트만 조회합니다. 발행에 실패한 이벤트에는 다음 시도 시각이 기록되지만, 보류된 후행 이벤트에는 기록되지 않았습니다. 다음 조회에서는 후행 이벤트만 선택되고, **배치 안에서만 순서를 검사하는 1차 수정**으로는 발행을 막을 수 없었습니다.

```
조회 조건 – 다음 시도 시각이 없거나 이미 지난 것

회차 1  [A-1, A-2]   A-1 발행 실패 → 시각을 12:00:05 로 밀어 둠
                     A-2 보류      → 시각 없음 (그대로)
회차 2  [     A-2]   A-1 은 시각이 미래라 빠지고, A-2 만 남아 그대로 발행 → 추월
```

선행 이벤트의 재시도 시각을 후행 이벤트에도 적용

```
A-1  발행 실패   다음 시도 시각 12:00:05
A-2  보류        다음 시도 시각 12:00:05   ← 앞의 시각을 그대로 받는다

회차 2        조회 → [        ]   둘 다 미래라 빠진다
12:00:05 뒤   조회 → [A-1, A-2]   같은 배치로 돌아와 1차 수정이 다시 걸린다
```

순서 역전 0건 — 문제 상황을 재현한 테스트 6건 · 처리량 ×1.13

두 결함 모두 **문제 상황을 재현하는 단위 테스트를 먼저 쓰고**, 그 테스트가 통과하도록 고쳤습니다. 여섯 건이 확인하는 조건은 아래와 같습니다.

`OutboxRelayTest` — 빌드마다 실행	통과 조건
`sameKeyIsPublishedInOrder`	같은 주문의 이벤트를 적재 순서대로 발행
`failureHoldsLaterEventsOfSameAggregate`	선행 이벤트 실패 시 같은 주문의 후행 이벤트를 보류
`failureOfOneKeyDoesNotBlockOtherKeys`	다른 주문의 이벤트는 계속 발행
`backOffDoesNotLetLaterEventOvertakeAcrossCycles`	선행 이벤트의 백오프 중에도 후행 이벤트 발행 방지
`heldEventDoesNotBurnRetryBudget`	선행 이벤트의 시도 횟수가 10회에 이를 때까지 후행 이벤트의 재시도 횟수는 0
`deadBlockerReleasesTheKey`	선행 이벤트의 발행을 중단하면 다음 회차부터 후행 이벤트 발행

수정 전에는 200건의 배치에서 브로커 응답을 200번 순차 대기했습니다. 주문 간 병렬 발행으로 대기 횟수가 대개 한 번으로 줄면서 배치 처리 시간은 0.47 → 0.28초, 처리량은 **138 → 156.1 events/s (×1.13)**로 개선됐습니다. 순서 보장 수정에 따른 개선은 이 ×1.13입니다.

Outbox에 미발행 이벤트가 남아 있어도 매 배치 뒤에 1초를 기다리는 동작은 남아 있었습니다. 폴링 주기는 1초로 유지하고, 한 번 실행할 때 200건씩 최대 10개 배치를 연속 발행하도록 바꿨습니다. 배치 사이의 대기를 줄여 처리량을 **156.1 → 480 events/s**로 높였습니다.

현재 순서 보장은 **같은 주문의 이벤트를 릴레이 한 대가 처리하는 조건**에서 성립합니다. 조회 질의가 주문번호별 분배를 보장하지 않아, 릴레이를 여러 대로 늘리면 같은 주문의 이벤트가 나뉘어 순서가 다시 역전될 수 있습니다. 확장 시에는 주문번호별 담당 릴레이를 지정해야 합니다. 현재는 릴레이 1대의 처리량이 480 events/s이고 API 한계가 132.5 RPS여서 릴레이 증설은 보류했습니다.

역전 0
빌드마다 재현 테스트 6건 실행

×1.13
주문 간 병렬 발행으로 처리량 개선

성능 저하의 원인을 확인하고 측정 환경·절차 개선

릴레이 개선이 HTTP API 성능에 미치는 영향을 부하 테스트로 확인했습니다. 개선 전 기준선은 74.1 RPS · p95 5.08s였지만, 개선 후에는 44.8 RPS · 10.0s로 저하됐습니다.
릴레이 변경이 성능 저하의 원인인지 확인하기 위해 추가 실험을 진행했습니다.

릴레이가 API와 커넥션 풀을 공유하므로, 릴레이를 비활성화한 상태에서도 측정했습니다. **릴레이 없이 실행한 회차도 44.9 RPS · p95 9.9s**로 비슷한 결과가 나왔습니다. 측정 환경의 메모리 부족으로 스와핑이 발생하고 있었으며, 실패 요청은 모두 커넥션 풀 고갈에 따른 것이었습니다.

메모리가 충분한 환경에서 재측정
순서 보장 수정은 유지하고, 스와핑이 발생하지 않는 환경으로 옮겼습니다. 측정 환경은 결과와 같은 폴더에 기록하고, 비교할 두 조건의 차이가 측정에 나타나는지 먼저 확인하도록 했습니다. 최대 처리량 측정과 개선 전후 비교도 별도 회차로 구분했습니다. Kafka에 컨슈머가 아직 처리하지 않은 이벤트 113,517건이 남아 있었지만, 소비 지연 지표는 0이었습니다. 브로커에서 조회한 컨슈머 그룹 오프셋으로 미처리 건수를 대조했습니다.

Outbox 발행 시 요청의 추적 정보 유실

아홉 서비스의 로그를 연결하기 위해 요청마다 식별자를 만들고 `[서비스명,식별자]` 형태로 기록했습니다. 서비스 간에는 Kafka 이벤트로 통신하므로, 같은 주문의 식별자가 order → payment → license → settlement까지 전달되는지 확인했습니다.

```
INFO [order,3f6c1d8a49b24e0f8c7a5b1e2d90f4a7]  주문 생성 orderNo=ORD20260809K7QF2M9XTB memberId=1041 amount=39000
INFO [payment,]                                결제 대기 생성 orderNo=ORD20260809K7QF2M9XTB amount=39000
INFO [license,]                                라이선스 지급 orderNo=ORD20260809K7QF2M9XTB products=[1001, 1002]
INFO [settlement,]                             매출 집계 orderNo=ORD20260809K7QF2M9XTB lines=2
```

order 이후 서비스에는 식별자가 기록되지 않았습니다. 추적 정보는 요청 스레드에만 있어 요청 종료 시 사라지지만, Kafka 발행은 최대 1초 뒤 릴레이에서 실행됐기 때문입니다.
Micrometer Tracing을 적용한 뒤에는 요청의 식별자 대신 **릴레이 폴링 회차의 traceId**가 기록됐습니다.

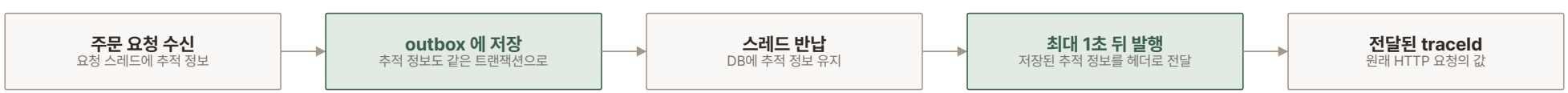


자동 계측은 발행 시점의 스레드에서 추적 정보를 가져옵니다. Outbox 릴레이는 요청 스레드 밖에서 실행되므로 **원래 요청의 추적 정보가 자동으로 전달되지 않았습니다.** 요청을 처리할 때의 추적 정보를 저장하고, 릴레이가 발행할 때 복원하는 방식이 필요했습니다.

추적 정보를 Outbox에 저장하고 발행 시 헤더로 전달

전달할 값은 트레이스 ID와 스팬 ID를 담은 W3C 추적 전파 헤더 `traceparent`입니다. 55자 고정 문자열인 이 값을 발행 시점까지 보존하기 위해 저장 위치를 비교했습니다. **이벤트와 같은 트랜잭션으로 저장할 수 있는지**를 기준으로 선택했습니다.

전달 방법	검토 결과
자동 계측에만 의존	한 배치의 서로 다른 주문이 릴레이의 traceId를 공유해 요청별 추적이 부정확해집니다
자체 헤더 구현	별도 규약을 유지해야 하며, 추적 라이브러리가 제공하는 구간별 소요 시간과 부모 · 자식 관계도 활용할 수 없습니다
이벤트 본문	이벤트 본문에 관측 도구 관련 필드가 추가되고, 본문 파싱 전에 헤더로 추적 정보를 확인하는 방식도 변경해야 합니다
릴레이에서 새 스팬 생성	발행 루프에서 스팬 수명을 관리해야 합니다. 발행 대기 시간은 기존 스팬 사이의 간격으로도 확인할 수 있습니다
채택 — outbox 컬럼 추가	이벤트와 추적 정보를 같은 행에 저장해 트랜잭션 단위로 함께 보존합니다. 이벤트 본문은 유지할 수 있지만, Outbox 테이블에 추적용 컬럼을 추가해야 합니다



릴레이가 나중에 발행해도 원래 요청의 추적 정보가 헤더로 전달됩니다. 여러 서비스를 거친 요청을 같은 traceId로 추적할 수 있습니다.

서비스 6곳의 추적 연결 후 +527ms의 폴링 대기 확인

```
gateway   http post                                  ROOT
└ payment  http post /api/v1/payments/callback        18ms
  ├ order       stove.payment.v1 receive             +527ms   ← Kafka 를 한 번 건넘
  ├ license     stove.payment.v1 receive             +527ms
  ├ settlement  stove.payment.v1 receive             +527ms
  └ (license 가 발행)
    ├ payment    stove.license.v1 receive            +759ms   ← 여기서 한 번 더
    └ download   stove.license.v1 receive            +759ms
```

결제 콜백 1건에서 수집한 스팬 트리입니다. 세 컨슈머의 부모 스팬이 **원래 HTTP 요청**으로 연결됐습니다. 세 컨슈머 모두 `+527ms` 뒤에 처리를 시작해, 서비스 지표만으로 확인하기 어려웠던 지연을 찾았습니다. 릴레이 대기 중에 저장된 이벤트는 다음 조회까지 기다려야 했습니다. 폴링 주기 1초에서 예상되는 평균 대기 0.5초와 비슷해 주기를 줄인 뒤 부하 테스트를 진행했습니다. **1000 → 200ms**로 변경하자 느린 쪽 5%(p95)의 종단 지연이 **절반 이하로 감소**했습니다. 조건마다 두 회차를 실행했으며, 아래 표에 회차별 결과와 평균을 함께 정리했습니다.

폴링 주기	회차 1 p95	회차 2 p95	p95 회차 평균	폴링이 만드는 대기
1000ms (기본값)	1.30s	1.06s	1.18s	0~1000ms 대기
200ms	563ms	561ms	562ms	0~200ms 대기

2026.08.10 · k6 5 VU·2분·조건별 2회. 승인 콜백 요청 직전부터 라이브러리 API의 권한 확인까지 측정했습니다. 지급 확인 간격은 0.5초이며, 전체 표본을 합친 p95가 아닌 회차별 p95의 평균입니다.

추적 일원화
서비스 6곳을 하나의 traceId로 연결

1.18s → 0.56s
콜백 요청 → 권한 확인 · 회차별 p95 평균

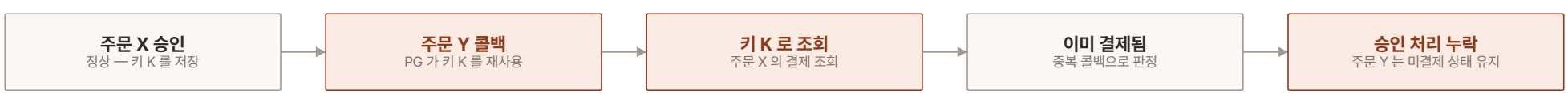
+1.5%p
MySQL CPU 7.3 → 8.8% · 폴링 빈도 5배 증가

멱등키로 결제를 조회해 다른 주문의 승인을 중복으로 오인

결제 대행사(PG)는 스텁으로 대체하고, 실제 연동을 모사한 3단계로 구성했습니다. 주문을 받으면 결제 대기 상태를 생성하고, PG에 금액을 사전등록해 거래 번호를 저장합니다. 사용자가 결제창에서 승인하면 PG가 보낸 콜백으로 결제를 확정합니다. 각 단계의 식별자는 다음과 같습니다.

단계	식별자	생성 · 저장 주체
결제 대기 생성	주문번호	서비스에서 생성
PG 사전등록	`pgTxId`	PG 응답을 받아 서비스에서 저장
승인 콜백 수신	멱등키	PG가 콜백 본문으로 전달

멱등키는 결제사가 생성해 콜백으로 전달합니다. **서로 다른 주문에 같은 키가 사용되지 않는다는 보장이 없었지만**, 결제 조회와 중복 판정, 테이블의 유니크 제약을 모두 이 키에 의존하도록 구현했습니다. 같은 키가 다른 주문에 사용됐을 때의 동작을 테스트로 확인했습니다.



PG 승인과 서비스의 결제 상태 불일치
결제 완료 이벤트가 발행되지 않아 라이선스 지급 · 주문 확정 · 정산이 누락됩니다.
PG에서는 승인이 끝났지만 서비스에는 미결제 상태로 남으며, 예외나 경고 로그도 발생하지 않습니다.

결제 조회·중복 판정·동시성 제어 분리

처음에는 멱등키로 결제를 조회해 결과가 있으면 이미 처리한 콜백으로 판단했습니다. 결제 조회와 중복 판정을 한 번에 처리하려고 작성한 코드입니다.

```
Payment payment = paymentRepository.findByIdempotencyKey(approval.idempotencyKey())
        .orElseGet(() -> findPayment(approval.orderNo()));
```

멱등키가 모든 주문에서 유일할 때만 동작하는 설계였습니다. 키가 재사용되면 다른 주문의 결제를 조회해 정상 승인을 중복으로 오인합니다. 조회 · 중복 판정 · 동시성 제어를 분리하고 각 역할에 맞는 수단을 적용했습니다.

역할	변경한 방식
조회	서비스가 생성하고 유일성을 보장하는 주문번호로 결제를 조회합니다
중복 판정	전체 결제에서 같은 키를 찾는 대신, 해당 결제에 이미 처리된 콜백인지 확인합니다
동시성	결제 행을 `SELECT … FOR UPDATE`로 잠가 같은 주문의 콜백을 순서대로 처리합니다
유니크 제약	다른 주문의 키가 같아도 저장할 수 있도록 일반 인덱스로 변경했습니다. 동시 콜백을 제어할 행 잠금을 먼저 적용한 뒤 유니크 제약을 제거했습니다

이미 승인된 주문에 같은 키의 콜백이 오면 재전송으로 판단해 중복 처리하지 않습니다. 키가 다르면 정상 재전송으로 볼 수 없으므로 **예외를 발생시킵니다**.

승인 콜백 입력 4종의 처리 결과 검증

콜백 입력을 네 가지로 나눠, 수정 전후에 각각 어떻게 처리되는지 테스트로 확인했습니다.

들어온 콜백	수정 전	수정 후
다른 주문에 사용된 키의 콜백	**미결제 상태로 승인 처리 누락**	해당 주문 승인
같은 콜백의 재전송	무시	중복 처리 방지 — 이벤트 한 번만 발행
승인된 주문에 다른 키의 승인	중복으로 판단해 무시	예외 발생
존재하지 않는 주문의 콜백	키가 같으면 다른 주문의 결제 조회	거부

결제 직전 상태의 주문을 준비하고 콜백 본문을 직접 구성해 테스트했습니다. 두 주문에 같은 키를 지정해 키 재사용 상황을 재현했습니다. 결제 상태와 발행된 이벤트 수를 함께 검증해 중복 발행도 확인했습니다. 거절 콜백에도 같은 검사를 적용했으며, 승인과 거절이 연이어 도착하면 먼저 확정된 상태를 유지하고 후속 콜백은 예외로 처리합니다.

스텁으로 검증한 범위
PG 스텁으로 같은 키의 콜백을 만들어 서비스의 처리 결과를 검증했습니다. 실제 결제사에서 키가 재사용되는 상황을 관측한 것은 아닙니다.

승인 유실 0
다른 주문과 같은 키여도 정상 승인

테스트 12건
승인 6 · 거절 6, 실제 MySQL에서 검증

콜백 2종 검증
승인·거절 충돌 시 후속 콜백 예외 처리

테스트 구성

단위 · 통합 · E2E 테스트 1,232건과 계층별 검증 범위

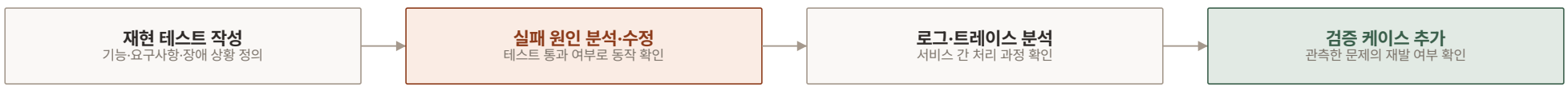
단위 테스트로 도메인부터 표현 계층까지의 비즈니스 규칙과 개별 동작을 검증했습니다.
멱등성 · 트랜잭션 경계 · 스키마는 실제 MySQL과 Kafka 컨테이너를 사용하는 통합 테스트로 확인했습니다.
E2E 테스트로 주문 → 결제 → 지급 → 정산의 API 흐름을 검증하고, AI를 활용한 코드 변경도 설계 규칙을 따르도록 **ArchUnit으로 계층 의존 방향과 모듈 경계를 빌드마다 검사**했습니다.



마무리

재현 테스트와 로그·트레이스 분석으로 확인한 문제

기능과 장애 상황을 테스트로 작성하고, 실패한 테스트를 기준으로 코드를 수정했습니다. 수정 후에도 같은 테스트를 빌드마다 실행해 재발 여부를 확인했습니다.
테스트에서 발견하지 못한 문제는 로그와 트레이스를 분석해 확인하고 검증 케이스를 추가했습니다. 사례 3에서는 트레이스에서 폴링 대기를 발견하고, 부하 테스트로 개선 효과를 측정했습니다.



사례	발견한 계기	개선 수치
1 정상 결제 자동 환불	`license` 의 DB 를 끊은 장애 주입 테스트	환불 8 → 0건
2 Outbox 발행 순서 역전	발행 재시도에 백오프를 적용하며 작성한 단위 테스트	역전 0건 · 처리량 ×1.13
3 Kafka 구간 추적 단절	같은 주문의 로그를 네 서비스에서 비교	추적 6곳 일원화 · p95 1.18s → 0.56s
4 중복으로 오인된 결제 승인	**같은 멱등키**를 두 주문에 사용한 테스트	승인 유실 0건